



Cyprus International University

Database Management Systems and Programing. II

2025-2026 Spring Term Project

Wildlife Conservation Monitoring Database Management System

Prepared By;

22208724 Mohamed Alfutahi

responsible from:

1. INTRODUCTION
2. Data Definition Language (DDL)

22118071 Mohamed Abdelgadir

responsible from:

3. ENTITY RELATIONSHIP DIAGRAM (ERD)

22204138 Sufyan Qaid

responsible from:

4. Data Manipulation Language (DML)

22318677 Osman Salih

responsible from:

5. PL/SQL QUERIES

22326910 Omar Awadh

responsible from:

6. USER INTERFACE(SOFTWARE)

Submitted to;

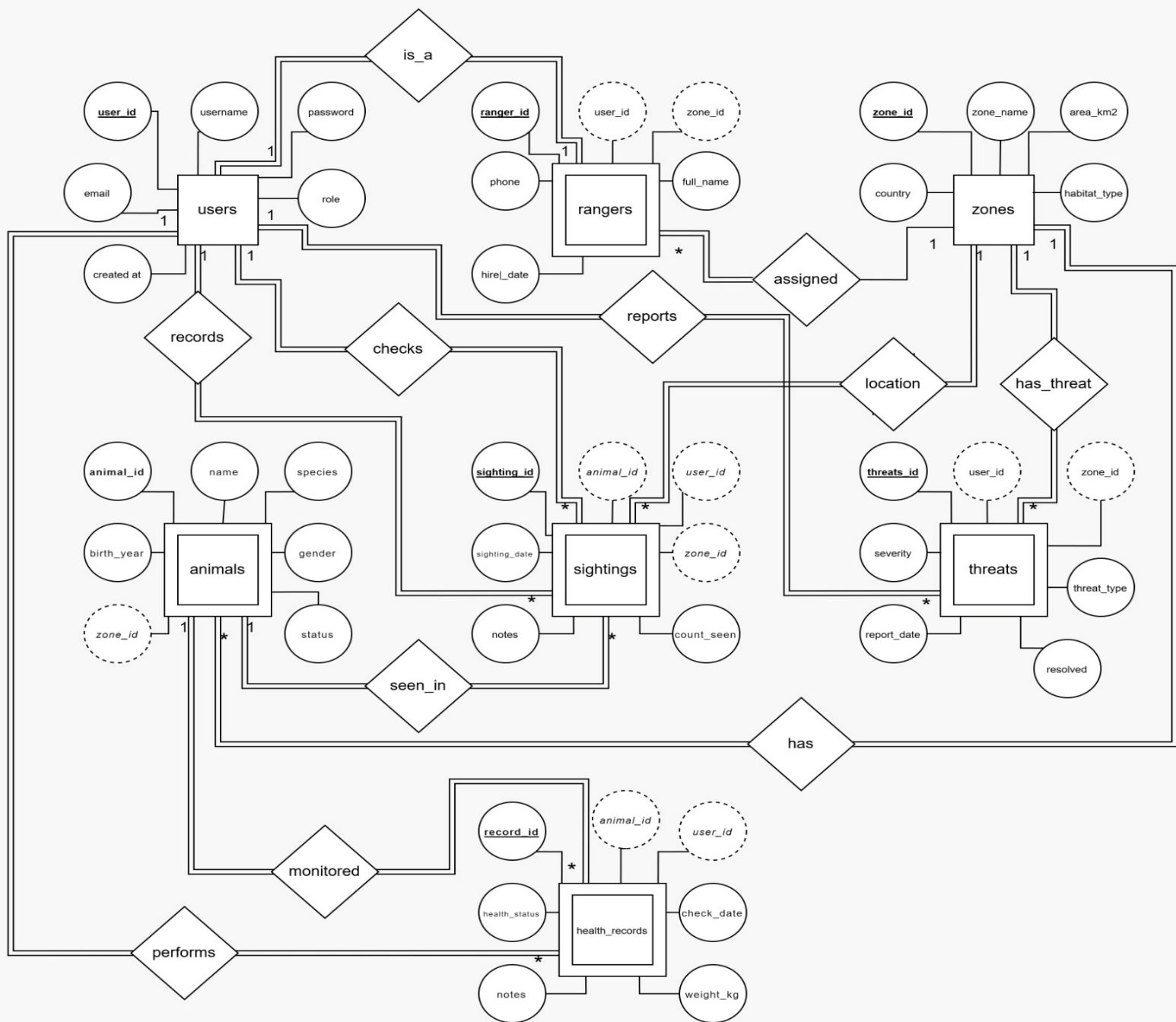
Dr. DİREKOĞLU Melike ŞAH

1. INTRODUCTION

The Wildlife Conservation Monitoring Database Management System is designed to help manage and monitor wildlife conservation activities in an organized way. The system stores important information about animals, protected zones, rangers, sightings, health records, threats, and users. It helps conservation staff track animal status, record sightings, monitor health conditions, and report threats such as poaching, fire, flood, drought, and disease.

This project assumes that each animal belongs to a protected zone, rangers are assigned to zones, and authorized users can record or view conservation data based on their roles. The database was implemented using PostgreSQL/Supabase, with proper tables, relationships, constraints, sample data, queries, functions, procedures, and triggers to make the system reliable and useful for real-world wildlife monitoring.

2. ENTITY RELATIONSHIP DIAGRAM



3. Data Definition Language (DDL)

Creating users table:

```
13 CREATE TABLE users (  
14     user_id      SERIAL PRIMARY KEY,  
15     username     VARCHAR(50) NOT NULL UNIQUE,  
16     password     VARCHAR(255) NOT NULL,  
17     email        VARCHAR(100) UNIQUE,  
18     role         VARCHAR(20) NOT NULL DEFAULT 'viewer'  
19     |           | CHECK (role IN ('admin','ranger','researcher','viewer')),  
20     created_at   TIMESTAMP DEFAULT NOW()  
21 );
```

Creating zones table:

```
3 CREATE TABLE zones (  
4     zone_id      SERIAL PRIMARY KEY,  
5     zone_name     VARCHAR(100) NOT NULL,  
6     area_km2     NUMERIC(10,2) CHECK (area_km2 > 0),  
7     habitat_type VARCHAR(50) NOT NULL DEFAULT 'forest'  
8     |           | CHECK (habitat_type IN ('forest','grassland','wetland','desert')),  
9     country      VARCHAR(100) NOT NULL DEFAULT 'Unknown'  
10 );
```

Creating animals table:

```
24 CREATE TABLE animals (  
25     animal_id   SERIAL PRIMARY KEY,  
26     name        VARCHAR(100) NOT NULL,  
27     species     VARCHAR(100) NOT NULL,  
28     gender      VARCHAR(10) DEFAULT 'unknown'  
29     |           | CHECK (gender IN ('male','female','unknown')),  
30     birth_year  INT          CHECK (birth_year > 1900 AND birth_year <= 2026),  
31     status      VARCHAR(20) NOT NULL DEFAULT 'alive'  
32     |           | CHECK (status IN ('alive','dead','unknown')),  
33     zone_id     INT REFERENCES zones(zone_id) ON DELETE SET NULL  
34 );
```

Creating rangers table:

```
37 CREATE TABLE rangers (  
38     ranger_id SERIAL PRIMARY KEY,  
39     user_id INT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
40     zone_id INT REFERENCES zones(zone_id) ON DELETE SET NULL,  
41     full_name VARCHAR(100) NOT NULL,  
42     phone VARCHAR(20),  
43     hire_date DATE DEFAULT CURRENT_DATE  
44 );
```

Creating sightings table:

```
47 CREATE TABLE sightings (  
48     sighting_id SERIAL PRIMARY KEY,  
49     animal_id INT NOT NULL REFERENCES animals(animal_id) ON DELETE CASCADE,  
50     user_id INT NOT NULL REFERENCES users(user_id),  
51     zone_id INT REFERENCES zones(zone_id),  
52     sighting_date DATE NOT NULL DEFAULT CURRENT_DATE,  
53     count_seen INT NOT NULL DEFAULT 1 CHECK (count_seen >= 1),  
54     notes TEXT  
55 );
```

Creating health_records table:

```
58 CREATE TABLE health_records (  
59     record_id SERIAL PRIMARY KEY,  
60     animal_id INT NOT NULL REFERENCES animals(animal_id) ON DELETE CASCADE,  
61     user_id INT NOT NULL REFERENCES users(user_id),  
62     check_date DATE NOT NULL DEFAULT CURRENT_DATE,  
63     health_status VARCHAR(20) NOT NULL DEFAULT 'healthy'  
64     | | | CHECK (health_status IN ('healthy', 'sick', 'injured', 'recovering')),  
65     weight_kg NUMERIC(8,2) CHECK (weight_kg > 0),  
66     notes TEXT  
67 );
```

Creating threats table:

```
70 CREATE TABLE threats (  
71     threat_id SERIAL PRIMARY KEY,  
72     zone_id INT NOT NULL REFERENCES zones(zone_id) ON DELETE CASCADE,  
73     reported_by INT NOT NULL REFERENCES users(user_id),  
74     threat_type VARCHAR(50) NOT NULL  
75     | | | CHECK (threat_type IN ('poaching','fire','flood','drought','disease')),  
76     severity VARCHAR(10) NOT NULL DEFAULT 'low'  
77     | | | CHECK (severity IN ('low','medium','high')),  
78     report_date DATE NOT NULL DEFAULT CURRENT_DATE,  
79     resolved BOOLEAN NOT NULL DEFAULT FALSE  
80 );
```

4. Data Manipulation Language (DML)

1. Insert users

```
97 INSERT INTO users (username, password, email, role) VALUES  
98 ('admin1', 'pass123', 'admin@wild.org', 'admin'),  
99 ('ranger_tom', 'pass123', 'tom@wild.org', 'ranger'),  
100 ('ranger_sara', 'pass123', 'sara@wild.org', 'ranger'),  
101 ('dr_james', 'pass123', 'james@wild.org', 'researcher'),  
102 ('viewer1', 'pass123', 'viewer1@wild.org', 'viewer');  
103
```

2.Insert zones

```
89 INSERT INTO zones (zone_name, area_km2, habitat_type, country) VALUES  
90 ('North Forest', 250.50, 'forest', 'Kenya'),  
91 ('South Plains', 180.00, 'grassland', 'Kenya'),  
92 ('East Wetlands', 95.30, 'wetland', 'Tanzania'),  
93 ('West Desert', 310.00, 'desert', 'Namibia'),  
94 ('Central Reserve',120.00, 'forest', 'Uganda');  
95
```

3.Insert animals

```

105 INSERT INTO animals (name, species, gender, birth_year, status, zone_id) VALUES
106 ('Leo', 'Lion', 'male', 2015, 'alive', 1),
107 ('Ella', 'Elephant', 'female', 2010, 'alive', 2),
108 ('Zara', 'Zebra', 'female', 2018, 'alive', 2),
109 ('Rocky', 'Rhino', 'male', 2008, 'alive', 3),
110 ('Mia', 'Cheetah', 'female', 2019, 'alive', 1),
111 ('Titan', 'Giraffe', 'male', 2012, 'unknown', 4),
112 ('Bora', 'Buffalo', 'male', 2016, 'alive', 5),
113 ('Lena', 'Leopard', 'female', 2017, 'alive', 1);
114

```

4.Insert rangers

```

116 INSERT INTO rangers (user_id, zone_id, full_name, phone, hire_date) VALUES
117 (2, 1, 'Tom Green', '+254700001111', '2020-03-15'),
118 (3, 2, 'Sara White', '+254700002222', '2021-06-01');
119

```

5.Insert sightings

```

121 INSERT INTO sightings (animal_id, user_id, zone_id, sighting_date, count_seen, notes) VALUES
122 (1, 2, 1, '2026-04-01', 1, 'Leo seen near the river'),
123 (2, 3, 2, '2026-04-02', 3, 'Group of elephants moving south'),
124 (3, 2, 2, '2026-04-03', 5, 'Zebra herd at the water hole'),
125 (4, 3, 3, '2026-04-04', 1, 'Rocky alone near the trees'),
126 (5, 2, 1, '2026-04-05', 2, 'Two cheetahs running fast'),
127 (1, 3, 1, '2026-04-06', 1, 'Leo again, looks strong'),
128 (6, 2, 4, '2026-04-07', 1, 'Titan spotted at sunrise'),
129 (7, 3, 5, '2026-04-08', 4, 'Buffalo group drinking water'),
130 (8, 2, 1, '2026-04-09', 1, 'Lena resting on a rock'),
131 (2, 2, 2, '2026-04-10', 2, 'Two elephants seen again');
132

```

6.Insert health records

```
134 INSERT INTO health_records (animal_id, user_id, check_date, health_status, weight_kg, notes) VALUES
135 (1, 4, '2026-04-01', 'healthy', 190.50, 'Good weight, teeth look normal'),
136 (2, 4, '2026-04-02', 'healthy', 4200.00, 'Very strong and active'),
137 (3, 4, '2026-04-03', 'healthy', 320.00, 'No problems found'),
138 (4, 4, '2026-04-04', 'injured', 1100.00, 'Cut on the left back leg'),
139 (5, 4, '2026-04-05', 'recovering', 55.30, 'Was sick last week, now better'),
140 (6, 4, '2026-04-06', 'healthy', 900.00, 'Titan looks good'),
141 (7, 4, '2026-04-07', 'sick', 620.00, 'Signs of fever, needs watching'),
142 (8, 4, '2026-04-08', 'healthy', 62.00, 'Lena is in great shape');
143
```

7.Insert threats

```
145 INSERT INTO threats (zone_id, reported_by, threat_type, severity, report_date, resolved) VALUES
146 (1, 2, 'poaching', 'high', '2026-04-01', FALSE),
147 (2, 3, 'drought', 'medium', '2026-04-02', FALSE),
148 (3, 2, 'flood', 'low', '2026-04-03', TRUE),
149 (4, 1, 'fire', 'high', '2026-04-04', FALSE),
150 (5, 3, 'disease', 'medium', '2026-04-05', FALSE);
151
```

5. Update Examples & Delete Examples

```
152 -- UPDATE examples
153 UPDATE animals SET status = 'dead' WHERE animal_id = 6;
154 UPDATE threats SET resolved = TRUE WHERE threat_id = 1;
155 UPDATE animals SET zone_id = 2 WHERE animal_id = 3;
156 UPDATE users SET password = 'newpass999' WHERE username = 'viewer1';
157
158 -- DELETE examples
159 DELETE FROM threats WHERE resolved = TRUE AND severity = 'low';
160 DELETE FROM sightings WHERE notes = 'test entry';
161
```

6. SQL QUERIES

Query 1: Total sightings per animal (JOIN + GROUP BY + COUNT + SUM)

Shows which animals are seen the most

```
3  SELECT
4      a.name                AS animal_name,
5      a.species,
6      COUNT(s.sighting_id)  AS total_sightings,
7      SUM(s.count_seen)     AS total_animals_seen
8  FROM animals a
9  JOIN sightings s ON a.animal_id = s.animal_id
10 GROUP BY a.name, a.species
11 ORDER BY total_sightings DESC;
```

Output:

animal_name	species	total_sightings	total_animals_seen
Leo	Lion	4	4
Ella	Elephant	2	5
Zara	Zebra	1	5
Rocky	Rhino	1	1
Titan	Giraffe	1	1
Mia	Cheetah	1	2
Lena	Leopard	1	1
Bora	Buffalo	1	4

Query 2: Number of animals per zone with habitat type (LEFT JOIN + GROUP BY)
Helps management see which zones have the most animals

```
3  SELECT
4      z.zone_name,
5      z.habitat_type,
6      z.country,
7      COUNT(a.animal_id)          AS animal_count
8  FROM zones z
9  LEFT JOIN animals a ON z.zone_id = a.zone_id
10 GROUP BY z.zone_name, z.habitat_type, z.country
11 ORDER BY animal_count DESC;
```

Output:

zone_name	habitat_type	country	animal_count
North Forest	forest	Kenya	3
South Plains	grassland	Kenya	2
Central Reserve	forest	Uganda	1
East Wetlands	wetland	Tanzania	1
West Desert	desert	Namibia	1

Query 3: Average weight per species from health records (JOIN + AVG + GROUP BY)
Shows the average weight of each species

```
3  SELECT
4      a.species,
5      ROUND(AVG(h.weight_kg), 2)    AS avg_weight_kg,
6      ROUND(MIN(h.weight_kg), 2)    AS min_weight_kg,
7      ROUND(MAX(h.weight_kg), 2)    AS max_weight_kg,
8      COUNT(h.record_id)            AS total_checks
9  FROM animals a
10 JOIN health_records h ON a.animal_id = h.animal_id
11 GROUP BY a.species
12 ORDER BY avg_weight_kg DESC;
```

Output:

species	avg_weight_kg	min_weight_kg	max_weight_kg	total_checks
Elephant	4200.00	4200.00	4200.00	1
Rhino	1100.00	1100.00	1100.00	1
Giraffe	900.00	900.00	900.00	1
Buffalo	620.00	620.00	620.00	1
Zebra	320.00	320.00	320.00	1
Lion	187.75	185.00	190.50	2
Leopard	62.00	62.00	62.00	1
Cheetah	55.30	55.30	55.30	1

Query 4: All unresolved high severity threats with zone info (JOIN + WHERE + ORDER BY)
Critical report for management: active dangers in each zone

```
3  SELECT
4      z.zone_name,
5      z.country,
6      t.threat_type,
7      t.severity,
8      t.report_date,
9      u.username                AS reported_by,
10     u.role
11  FROM threats t
12  JOIN zones z ON t.zone_id    = z.zone_id
13  JOIN users u ON t.reported_by = u.user_id
14  WHERE t.severity IN ('high','medium')
15         AND t.resolved = FALSE
16  ORDER BY t.report_date DESC;
```

Query 5: Search all sightings by a specific user ID (required from project notes)
Replace 2 with any user_id to search

```
3  SELECT
4      u.username,
5      u.role,
6      a.name                AS animal_name,
7      a.species,
8      s.sighting_date,
9      s.count_seen,
10     s.notes,
11     z.zone_name
12  FROM sightings s
13  JOIN users  u ON s.user_id    = u.user_id
14  JOIN animals a ON s.animal_id = a.animal_id
15  JOIN zones  z ON s.zone_id    = z.zone_id
16  WHERE u.user_id = 2
17  ORDER BY s.sighting_date DESC;
```

Query 6: Search sightings by zone name / address (required from project notes)
Replace 'forest' with any zone name or part of a name

```
3  SELECT
4      z.zone_name,
5      z.habitat_type,
6      a.name                AS animal_name,
7      a.species,
8      s.sighting_date,
9      s.count_seen,
10     u.username            AS reported_by
11 FROM sightings s
12 JOIN zones    z ON s.zone_id  = z.zone_id
13 JOIN animals  a ON s.animal_id = a.animal_id
14 JOIN users    u ON s.user_id   = u.user_id
15 WHERE LOWER(z.zone_name) LIKE LOWER('%forest%')
16 ORDER BY s.sighting_date DESC;
```

Output:

zone_name	habitat_type	animal_name	species	sighting_date	count_seen	reported_by
North Forest	forest	Leo	Lion	2026-05-11	1	ranger_tom
North Forest	forest	Leo	Lion	2026-05-11	1	ranger_tom
North Forest	forest	Lena	Leopard	2026-04-09	1	ranger_tom
North Forest	forest	Leo	Lion	2026-04-06	1	ranger_sara
North Forest	forest	Mia	Cheetah	2026-04-05	2	ranger_tom
North Forest	forest	Leo	Lion	2026-04-01	1	ranger_tom

Query 7: Animals not seen in the last 30 days -- conservation alert (subquery)

Finds alive animals with no recent sighting -- needs attention

```
3  SELECT
4      a.animal_id,
5      a.name,
6      a.species,
7      a.status,
8      z.zone_name
9  FROM animals a
10 LEFT JOIN zones z ON a.zone_id = z.zone_id
11 WHERE a.animal_id NOT IN (
12     SELECT DISTINCT animal_id
13     FROM sightings
14     WHERE sighting_date >= CURRENT_DATE - INTERVAL '30 days'
15 )
16 AND a.status = 'alive'
17 ORDER BY a.name;
```

Output:

animal_id	name	species	status	zone_name
7	Bora	Buffalo	alive	Central Reserve
2	Ella	Elephant	alive	South Plains
8	Lena	Leopard	alive	North Forest
5	Mia	Cheetah	alive	North Forest
4	Rocky	Rhino	alive	East Wetlands
3	Zara	Zebra	alive	South Plains

7. PL/SQL QUERIES

1.PROCEDURE: Search sightings by user ID

```
4 CREATE OR REPLACE PROCEDURE get_sightings_by_user(p_user_id INT)
5 LANGUAGE plpgsql AS $$
6 DECLARE
7     v_username VARCHAR;
8     v_count     INT;
9 BEGIN
10     SELECT username INTO v_username
11     FROM users WHERE user_id = p_user_id;
12
13     IF NOT FOUND THEN
14         RAISE NOTICE 'No user found with ID = %', p_user_id;
15         RETURN;
16     END IF;
17
18     SELECT COUNT(*) INTO v_count
19     FROM sightings WHERE user_id = p_user_id;
20
21     RAISE NOTICE 'User: % | Total sightings reported: %', v_username, v_count;
22 END;
23 $$;
24
25 CALL get_sightings_by_user(2);
```

2.PROCEDURE: Search animals by zone name (address)

```
5 CREATE OR REPLACE PROCEDURE search_by_zone(p_zone_name VARCHAR)
6 LANGUAGE plpgsql AS $$
7 DECLARE
8     v_zone_id     INT;
9     v_zone_name   VARCHAR;
10    v_count        INT;
11 BEGIN
12     SELECT zone_id, zone_name INTO v_zone_id, v_zone_name
13     FROM zones
14     WHERE LOWER(zone_name) LIKE LOWER('%' || p_zone_name || '%')
15     LIMIT 1;
16
17     IF NOT FOUND THEN
18         RAISE NOTICE 'No zone found matching: %', p_zone_name;
19         RETURN;
20     END IF;
21
22     SELECT COUNT(*) INTO v_count
23     FROM animals WHERE zone_id = v_zone_id;
24
25     RAISE NOTICE 'Zone found: % | Animals living here: %', v_zone_name, v_count;
26 END;
27 $$;
28
29 CALL search_by_zone('Forest');
```

3.PROCEDURE: Add a new sighting record

```
4 CREATE OR REPLACE PROCEDURE add_sighting(  
5     p_animal_id INT,  
6     p_user_id   INT,  
7     p_zone_id   INT,  
8     p_count     INT,  
9     p_notes     TEXT  
10 )  
11 LANGUAGE plpgsql AS $$  
12 DECLARE  
13     v_animal_name VARCHAR;  
14 BEGIN  
15     -- Check the animal exists  
16     SELECT name INTO v_animal_name  
17     FROM animals WHERE animal_id = p_animal_id;  
18  
19     IF NOT FOUND THEN  
20         RAISE NOTICE 'Animal with ID % does not exist.', p_animal_id;  
21         RETURN;  
22     END IF;  
23  
24     -- Insert the sighting  
25     INSERT INTO sightings (animal_id, user_id, zone_id, count_seen, notes)  
26     VALUES (p_animal_id, p_user_id, p_zone_id, p_count, p_notes);  
27  
28     RAISE NOTICE 'Sighting added for animal: % (ID = %)', v_animal_name, p_animal_id;  
29 END;  
30 $$;
```

4.FUNCTION: Count total sightings for one animal

```
4 CREATE OR REPLACE FUNCTION count_animal_sightings(p_animal_id INT)  
5 RETURNS INT  
6 LANGUAGE plpgsql AS $$  
7 DECLARE  
8     v_count INT;  
9 BEGIN  
10     SELECT COUNT(*) INTO v_count  
11     FROM sightings  
12     WHERE animal_id = p_animal_id;  
13  
14     RETURN v_count;  
15 END;  
16 $$;  
17  
18 SELECT count_animal_sightings(1) AS leo_sighting_count;  
19
```

Results Explain Chart Export ▾



Source

Primary Database ▾

leo_sighting_count

6

5. TRIGGER:

Auto-update animal status after health check When a health record is inserted with status 'sick' or 'injured', the trigger automatically changes the animal status to 'unknown'

```

6 CREATE OR REPLACE FUNCTION update_animal_status_on_health()
7 RETURNS TRIGGER
8 LANGUAGE plpgsql AS $$
9 DECLARE
10     v_animal_name VARCHAR;
11 BEGIN
12     SELECT name INTO v_animal_name
13     FROM animals WHERE animal_id = NEW.animal_id;
14
15     IF NEW.health_status IN ('sick', 'injured') THEN
16         UPDATE animals
17         SET status = 'unknown'
18         WHERE animal_id = NEW.animal_id;
19
20         RAISE NOTICE 'TRIGGER: Animal "%" status changed to unknown due to health status: %',
21             v_animal_name, NEW.health_status;
22     END IF;
23
24     RETURN NEW;
25 END;
26 $$;
27
28 CREATE TRIGGER trg_animal_health_update
29 AFTER INSERT ON health_records
30 FOR EACH ROW
31 EXECUTE FUNCTION update_animal_status_on_health();

```

Test it:

```
34 INSERT INTO health_records (animal_id, user_id, check_date, health_status, weight_kg, notes)
35 VALUES (1, 4, CURRENT_DATE, 'injured', 185.0, 'Leo has a wound on right leg');
36 SELECT animal_id, name, status FROM animals WHERE animal_id = 1;
```

Results	Explain	Chart	Export ▾		Source	Primary Database ▾	Role postgres ▾	Run Ctrl+R
animal_id	name	status						
1	Leo	unknown						

8. USER INTERFACE(SOFTWARE)

The user interface was developed using HTML, CSS, and JavaScript. The system is connected to the Supabase PostgreSQL database using the Supabase JavaScript client. The interface allows authorized users to log in, view records, add new records, edit existing records, delete records, and display management reports from the database.

The design follows a consistent dashboard style with a sidebar navigation menu, top bar, statistic cards, data tables, search/filter tools, and add/edit forms. The system includes role-based access control, where each user can access pages according to their role.

Database Connection Code

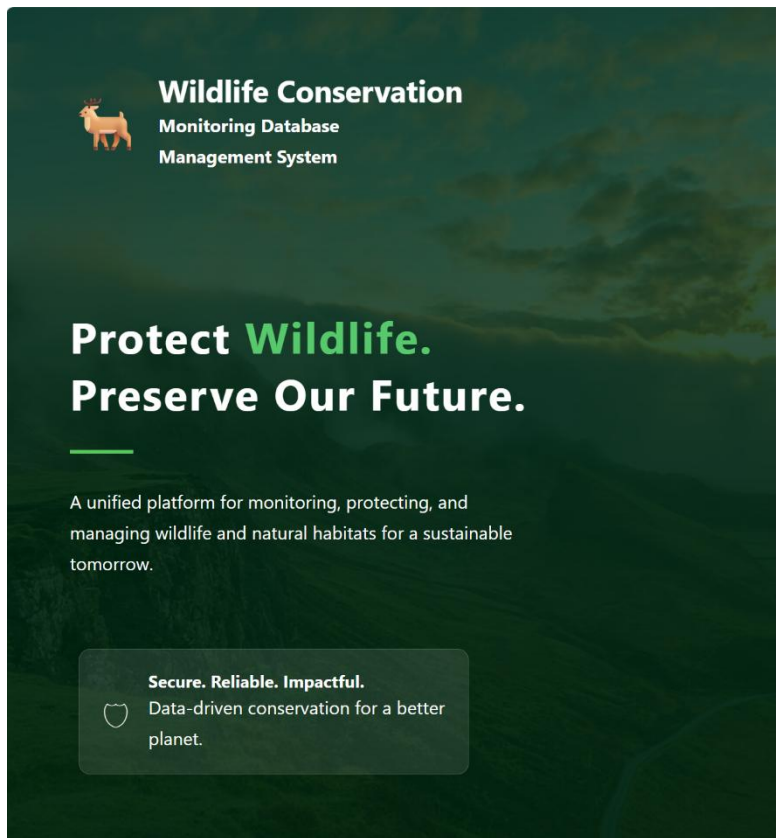
```
/* SUPABASE INFORMATION */
const SUPABASE_URL = "https://ygvrgfrghzccyatqhlww.supabase.co";
const SUPABASE_ANON_KEY = "sb_publishable_yZKp9A90eVXmybrzRrYQKw_Gb4EGd-1";

let supabaseClient = null;

if (
  typeof supabase !== "undefined" &&
  SUPABASE_URL &&
  SUPABASE_ANON_KEY &&
  !SUPABASE_ANON_KEY.includes("PASTE")
) {
  supabaseClient = supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
}
```

Login Page

The login page allows only registered database users to access the system. The user must enter username or email, password, and role. The system checks these values against the users table in Supabase. If the login information is correct, the user is redirected to the correct page based on the user role.





Welcome Back!

Only database users can access the system

Username or Email

Enter username or email

Password

Enter password

Role

Select your role

Login →

Login must match username/email, password.

Dashboard Page

The dashboard page gives an overview of the whole system. It displays total animals, health records, sightings, threats, rangers, users, and zones. It also shows recent activity, sightings over time, threats by severity, recent threats, zones overview, and recently added animals. The dashboard data is loaded dynamically from Supabase.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports



omar
admin

Welcome back, omar

Wildlife Conservation Monitoring Database Management System

8
Animals

10
Health Records

14
Sightings

5
Threats

3
Rangers

7
Users

5
Zones

Recent Activity

 Ranger added
syfuan • by omar

May 17, 2026

 User added
stu • by omar

May 17, 2026

 User deleted
stu • by omar

May 17, 2026

 User added
stu • by omar

May 17, 2026

 Ranger edited
Sara White • by omar

May 17, 2026

Sightings Over Time

Apr 6

1

Apr 7

1

Apr 8

1

Apr 9

1

Apr 10

1

May 11

2

May 17

2

Zones Overview

Zone Name	Area	Habitat
North Forest	250.5	forest
South Plains	180	grassland
East Wetlands	95.3	wetland
West Desert	310	desert
Central Reserve	120	forest

Top Animals

Animal	Species	Status	Birth Year
Lena	Leopard	alive	2017
Bora	Buffalo	alive	2016
Titan	Giraffe	dead	2012
Mia	Cheetah	alive	2019
Rocky	Rhino	alive	2008

Threats by Severity

Total Threats

5

Resolved

2

High

3

Medium

2

Low

0

Resolved

2

Recent Threats

Type	Zone	Severity	Date	Resolved
disease	Central Reserve	medium	2026-04-05	✕
fire	West Desert	high	2026-04-04	✕
drought	South Plains	medium	2026-04-02	✕
poaching	North Forest	high	2026-04-01	✓
fire	North Forest	high	2025-10-05	✓

© 2026 Wildlife Conservation Monitoring Database Management System.

WCMDMS v1.0.0

Animals Page

The Animals page allows the admin to manage animal records. The user can view animals, search by name or species, add a new animal, edit animal information, and delete animal records. The form includes animal name, species, gender, birth year, status, and zone.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports



omar
admin

Animals

Manage and track all registered animals in the system.

8
Total Animals
Registered

4
Male
Animals

4
Female
Animals

6
Alive
Animals

1
Dead
Animals

5
Zones
Represented

Search by name, species...

Filter

Reset

Animals List

View and manage all animals.

Export

ID	Name	Species	Gender	Birth Year	Status	Zone	Actions
1	Leo	Lion	male	2015	unknown	North Forest	 
2	Ella	Elephant	female	2010	alive	South Plains	 
3	Zara	Zebra	female	2018	alive	South Plains	 
4	Rocky	Rhino	male	2008	alive	East Wetlands	 
5	Mia	Cheetah	female	2019	alive	North Forest	 
6	Titan	Giraffe	male	2012	dead	West Desert	 
7	Bora	Buffalo	male	2016	alive	Central Reserve	 
8	Lena	Leopard	female	2017	alive	North Forest	 

Add / Edit Animal

Create a new animal record or update details.

Animal ID

Auto-generated

Name *

Enter animal name

Species *

Enter species

Gender *

Select gender

Birth Year *

YYYY

Status *

Select status

Zone *

Select zone

Save Animal

Clear

© 2026 Wildlife Conservation Monitoring Database Management System.

WCMDMS v1.0.0

Health Records Page

The Health Records page stores health check information for animals. The user can add health status, check date, weight, and notes. When a health record is inserted with status sick or injured, the database trigger automatically updates the animal status to unknown.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports

 **omar**
admin

Health Records 

Manage and track the health check records of all registered animals.

10
Total Records

5
Healthy

3
Injured

1
Recovering

1
Sick

Search by animal name, notes...

Clear Filters

Health Records

record_id	Animal	check_date	health_status	weight_kg	notes	Actions
1	Leo	2026-04-01	healthy	190.5	Good weight, teeth look normal	
2	Ella	2026-04-02	healthy	4200	Very strong and active	
3	Zara	2026-04-03	healthy	320	No problems found	
4	Rocky	2026-04-04	injured	1100	Cut on the left back leg	
5	Mia	2026-04-05	recovering	55.3	Was sick last week, now better	
6	Titan	2026-04-06	healthy	900	Titan looks good	
7	Bora	2026-04-07	sick	620	Signs of fever, needs watching	
8	Lena	2026-04-08	healthy	62	Lena is in great shape	
9	Leo	2026-05-11	injured	185	Leo has a wound on right leg	
12	Leo	2026-05-17	injured	185	Leo has a wound on right leg	

 **Add / Edit Health Record**

Record ID

Auto-generated

Animal

Select an animal

Check Date

mm/dd/yyyy

Health Status

Select status

Weight (kg)

Enter weight in kg

Notes

Enter notes about the health check

Cancel

Save Record

Sightings Page

The Sightings page records animal sightings. The user selects the animal, zone, reporter, sighting date, and notes. The table displays readable names such as animal name, zone name, and reported user instead of only foreign key IDs.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports

omar
admin

Sightings

Manage animal sightings recorded by rangers.

Search sightings...

Filter

Sightings List

sighting_id	Animal	Zone	Reported By	sighting_date	notes	Actions
1	Leo	North Forest	ranger_tom	2026-04-01	Leo seen near the river	
2	Ella	South Plains	ranger_sara	2026-04-02	Group of elephants moving south	
3	Zara	South Plains	ranger_tom	2026-04-03	Zebra herd at the water hole	
4	Rocky	East Wetlands	ranger_sara	2026-04-04	Rocky alone near the trees	
5	Mia	North Forest	ranger_tom	2026-04-05	Two cheetahs running fast	
6	Leo	North Forest	ranger_sara	2026-04-06	Leo again, looks strong	
7	Titan	West Desert	ranger_tom	2026-04-07	Titan spotted at sunrise	
8	Bora	Central Reserve	ranger_sara	2026-04-08	Buffalo group drinking water	
9	Lena	North Forest	ranger_tom	2026-04-09	Lena resting on a rock	
10	Ella	South Plains	ranger_tom	2026-04-10	Two elephants seen again	
11	Leo	North Forest	ranger_tom	2026-05-11	Leo seen near the north river at 7am	
12	Leo	North Forest	ranger_tom	2026-05-11	Leo seen near the north river at 7am	
14	Leo	North Forest	ranger_tom	2026-05-17	Leo seen near the north river at 7am	
15	Leo	North Forest	ranger_tom	2026-05-17	Leo seen near the north river at 7am	

Add / Edit Sighting

sighting_id

Auto-generated

animal_id

Select animal

zone_id

Select zone

reported_by

Select reporter

sighting_date

mm/dd/yyyy

notes

Enter sighting notes

Cancel

Save Sighting

Threats Page

The Threats page allows users to record threats such as poaching, drought, fire, flood, and disease. Each threat includes zone, reported user, threat type, severity, report date, and resolved status. This helps management monitor active conservation risks.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports

omar
admin

Threats Management

Monitor and manage threats to wildlife and their habitats.

May 17, 2026

Logout

5
Total Threats

3
High Severity

2
Medium Severity

2
Resolved

4
Zones Affected

Search threats...

Filters

Threats

Export

threat_id	zone_id	reported_by	threat_type	severity	report_date	resolved	Actions
1	North Forest	ranger_tom	poaching	high	2026-04-01	TRUE	
2	South Plains	ranger_sara	drought	medium	2026-04-02	FALSE	
4	West Desert	admin1	fire	high	2026-04-04	FALSE	
5	Central Reserve	ranger_sara	disease	medium	2026-04-05	FALSE	
7	North Forest	omar	fire	high	2025-10-05	TRUE	

Add / Edit Threat

threat_id

Auto-generated

zone_id

Select Zone

reported_by

Select Reporter

threat_type

Select Threat Type

severity

Select Severity

report_date

mm/dd/yyyy

resolved

Select Status

Reset

Save Threat

Zones Page

The Zones page manages conservation zones. It stores zone name, area, habitat type, and country. The page also displays summary information about habitat distribution and number of zones.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports

Zones

5
Total Zones

2
Forest Zones

1
Wetland Zones

1
Grassland Zones

1
Desert Zones

4
Countries

Search by zone_id, zone_name, habitat_type, country...

Filters

Zones

zone_id	zone_name	area_km2	habitat_type	country	Actions
1	North Forest	250.5	forest	Kenya	 
2	South Plains	180	grassland	Kenya	 
3	East Wetlands	95.3	wetland	Tanzania	 
4	West Desert	310	desert	Namibia	 
5	Central Reserve	120	forest	Uganda	 

Add / Edit Zone

zone_id

Auto-generated

zone_name

Enter zone name

area_km2

Enter area

habitat_type

Select habitat type

country

Enter country

Cancel

Save Zone

© 2026 Wildlife Conservation Monitoring Database Management System.

WCMDMS v1.0.0

Rangers Page

The Rangers page manages ranger assignments. A ranger account is first created in the Users & Roles page with role ranger, then selected in the Rangers page. The page stores the ranger user ID, assigned zone, full name, phone number, and hire date.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports

Rangers

3
Total Rangers

2
Active Zones Covered

Search by ranger_id, full_name, phone...

Filter

Sort

Rangers

ranger_id	User ID	zone_id	full_name	phone	hire_date	Actions
1	2	North Forest	Tom Green	+254700001111	2020-03-16	 
2	3	South Plains	Sara White	+254700002222	2021-07-01	 
3	6	North Forest	syfuan	+254700661111	2026-06-12	 

Add / Edit Ranger

ranger_id

Auto-generated

Ranger User

Select ranger user

zone_id

Select zone

full_name

Enter full name

phone

Enter phone number

hire_date

mm/dd/yyyy

Cancel

Save Ranger

© 2026 Wildlife Conservation Monitoring Database Management System.

WCMDMS v1.0.0

Users & Roles Page

The Users & Roles page manages system users and role permissions. Admin can create users with roles such as admin, ranger, researcher, and viewer. These roles control which pages the user can access and whether the user can add, edit, or delete records.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports

Users & Roles 

Manage system users, assign roles, and control access permissions.

Role Summary

admin
2
user

ranger
3
users

researcher
1
user

viewer
1
user

 Users



Search users by username, email, or role...

Filters

User ID	Username	Email	Role	Created At	Status	Actions
1	admin1	admin@wild.org	admin	2026-05-05T01:36:35.448046	 	
2	ranger_tom	tom@wild.org	ranger	2026-05-05T01:36:35.448046	 	
3	ranger_sara	sara@wild.org	ranger	2026-05-05T01:36:35.448046	 	
4	dr_james	james@wild.org	researcher	2026-05-05T01:36:35.448046	 	
5	viewer1	viewer1@wild.org	viewer	2026-05-05T01:36:35.448046	 	
6	omar	ome1646@gmail.com	admin	2026-05-05T00:56:21	 	
8	sfu	ome622097@gmail.com	ranger	2026-05-17T16:07:29.423	 	

Create / Edit User 

User ID

Auto-generated

Username *

Enter username

Email *

Enter email address

Role *

Select role

Password *

Enter password

Confirm Password *

Confirm password

Cancel

Save User

 Password Policy

Passwords must be at least 8 characters .

Reports Page

The Reports page displays management and statistical information from the database. The user can click report buttons to display clear query results in a table. These reports help management understand animal distribution, health trends, threats, sightings, and conservation alerts.



WCMDMS
Wildlife Conservation
Monitoring Database
Management System

Dashboard

Animals

Health Records

Sightings

Threats

Rangers

Zones

Users & Roles

Reports

Reports

Run management reports and database function/procedure operations.

8
Animals
Total registered

14
Sightings
Total records

5
Threats
Total reports

5
Zones
Conservation areas

Available Management Reports

Click any report button to display clear management information below.

Animals per Zone

Average Weight per Species

Unresolved Threats

Sightings by Zone

Animals Not Seen in 30 Days

Sightings by Ranger

Total Sightings per Animal

Report Explanation

Select a report from the left side. The result will be displayed in a clear table.

PL/pgSQL Functions / Procedures Buttons

Search Sightings by User ID

Calls a database function to count how many sightings were reported by a selected user.

Enter user_id, example: 2

Run Function

Search Animals by Zone

Calls a database function to search zones by name and count animals inside matching zones.

Enter zone name, example: Forest

Run Function

Count Animal Sightings

Calls a database function to count how many times a specific animal was seen.

Enter animal_id, example: 1

Run Function

Report Result

No report selected yet.

Export

ommar
admin

PL/pgSQL Functions / Procedures Buttons

Search Sightings by User ID

Calls a database function to count how many sightings were reported by a selected user.

1

Run Function

Search Animals by Zone

Calls a database function to search zones by name and count animals inside matching zones.

Enter zone name, example: Forest

Run Function

Count Animal Sightings

Calls a database function to count how many times a specific animal was seen.

Enter animal_id, example: 1

Run Function

Procedure Result: Search Sightings by User ID

The procedure searches for a user by ID, then counts how many sightings were reported by that user.

Export

User ID	Username	Total Sightings Reported
1	admin1	0

PL/pgSQL Functions / Procedures Buttons

Search Sightings by User ID

Calls a database function to count how many sightings were reported by a selected user.

1

Run Function

Search Animals by Zone

Calls a database function to search zones by name and count animals inside matching zones.

Forest

Run Function

Count Animal Sightings

Calls a database function to count how many times a specific animal was seen.

Enter animal_id, example: 1

Run Function

Procedure Result: Search Animals by Zone

The procedure searches for a zone by name and counts how many animals live in that zone.

Export

Zone ID	Zone Name	Habitat Type	Country	Animals Living Here
1	North Forest	forest	Kenya	3

PL/pgSQL Functions / Procedures Buttons

Search Sightings by User ID

Calls a database function to count how many sightings were reported by a selected user.

1

Run Function

Search Animals by Zone

Calls a database function to search zones by name and count animals inside matching zones.

Forest

Run Function

Count Animal Sightings

Calls a database function to count how many times a specific animal was seen.

1

Run Function

Function Result: Count Animal Sightings

This result comes from the database function count_animal_sightings().

Export

Animal ID	Total Sightings
1	6

9. Github

GitHub repository link:

<https://github.com/sufyanqaid2/Wildlife-Conservation-Monitoring--DMBS-Project.git>